PuntoFedele (Fidelity Card) – Documentazione delle API n8n

Project Work IFTS · Flutter Web → n8n → MongoDB

Questo documento descrive le **6 API** realizzate in n8n per il progetto *Sistema di raccolta punti per un negozio* (Project Work 2): cosa fa ciascun workflow, **nodo per nodo**, come sono fatti i dati su MongoDB e **quali chiamate eseguire da Postman** per verificarne il funzionamento, con le risposte attese.

Come usare le chiamate. In Postman: metodo `POST`, scheda *Body* → *raw* → *JSON*, URL `http://localhost:5678/webhook/<path>`. I body riportati nei test sono pensati per essere **copiati dal file Markdown**: nel PDF le righe lunghe vanno a capo e potrebbero introdurre interruzioni. Dove compare un segnaposto come `ID_MOVIMENTO` va sostituito con l'ID reale ricevuto in una risposta precedente.

Indice

1. Panoramica
2. Dati su MongoDB
3. Concetti ricorrenti nei workflow
4. API 01 – Login e verifica permessi
5. API 02 – Ricerca cliente
6. API 03 – Registra acquisto
7. API 04 – Storico movimenti e annullamento
8. API 05 – Catalogo premi e riscatto
9. API 06 – Dashboard
10. Percorso di test integrato
11. Limiti noti e semplificazioni didattiche
12. Risoluzione dei problemi
13. Mappa requisiti → API

---

1. Panoramica

1.1 Architettura



Flutter **non accede mai direttamente a MongoDB**: parla solo con le API esposte da n8n. Ogni workflow è un endpoint `POST` che riceve un JSON, esegue controlli e operazioni sul database e risponde con un JSON.

1.2 Gli endpoint

N.	Workflow	Metodo e path	Chi può usarla	Collection usate
01	Login e verifica permessi	`POST /webhook/login`	tutti (è l'ingresso)	`users` (lettura)
02	Ricerca cliente	`POST /webhook/ricerca-cliente`	qualunque utente autenticato	`clients`, `purchases_movements` (lettura)
03	Registra acquisto	`POST /webhook/registra-acquisto`	qualunque utente autenticato	`clients` (lettura e scrittura), `purchases_movements` (scrittura)
04	Storico movimenti e annullamento	`POST /webhook/movimenti`	storico: tutti; annullamento: **solo Amministratore**	`clients`, `purchases_movements`
05	Catalogo premi e riscatto	`POST /webhook/premi`	**solo Amministratore**	`clients`, `rewards_catalog`, `purchases_movements`
06	Dashboard	`POST /webhook/dashboard`	**Responsabile e Amministratore**	`clients`, `purchases_movements` (lettura)

Gli URL completi hanno la forma `http://localhost:5678/webhook/<path>` (workflow **pubblicato**, URL di produzione). Con il workflow non pubblicato e il pulsante *Execute workflow* si usa invece `http://localhost:5678/webhook-test/<path>`, che risponde a una sola chiamata.

1.3 Ruoli e permessi

Funzione	Operatore	Responsabile	Amministratore	Controllo fatto dall'API?
Login	✔	✔	✔	–
Cercare un cliente, vedere saldo e movimenti	✔	✔	✔	no (aperta a tutti i ruoli)
Registrare un acquisto	✔	✔	✔	no (aperta a tutti i ruoli)

Funzione	Operatore	Responsabile	Amministratore	Controllo fatto dall'API?
Consultare lo storico	✔	✔	✔	no (aperta a tutti i ruoli)
Dashboard	✘	✔	✔	**sì** (06)
Annullare un'operazione	✘	✘	✔	**sì** (04)
Catalogo premi e riscatto	✘	✘	✔	**sì** (05)

Le funzioni riservate (annullamento, premi, dashboard) sono protette **anche dalle API**, non solo dall'interfaccia: nascondere un pulsante in Flutter non basta, perché la chiamata potrebbe arrivare comunque. Le API aperte a tutti i ruoli non hanno bisogno di un controllo.

1.4 Formato delle risposte e codici HTTP

Le risposte sono sempre JSON con il campo `success` (`true` / `false`). In caso di errore compaiono anche `errore` (codice leggibile dal programma) e `messaggio` (testo per l'utente). L'API 01 usa solo `success` e `messaggio`.

```
{
  "success": false,
  "errore": "CLIENTE_NON_TROVATO",
  "messaggio": "Il cliente richiesto non esiste"
}
```

Codice HTTP	Significato in questo progetto	Esempio di `errore`
200	operazione riuscita	–
400	dati mancanti o non validi	`INPUT_NON_VALIDO`
401	credenziali errate (solo login)	–
403	operazione non autorizzata per il ruolo	`NON_AUTORIZZATO`
404	record inesistente	`CLIENTE_NON_TROVATO`, `MOVIMENTO_NON_ANNULLABILE`, `PREMIO_NON_DISPONIBILE`
409	conflitto con lo stato dei dati	`MOVIMENTO_GIA_ANNULLATO`, `SALDO_INSUFFICIENTE`

2. Dati su MongoDB

Database: `fidelity_db`. Nome esatto della collection dei movimenti: `purchases_movements` (con la *s*).

2.1 Collection `users`

Campo	Tipo	Note
`_id`	ObjectId	
`username`	string	usato per il login
`password`	string	**in chiaro** (semplificazione didattica ammessa dal progetto)
`ruolo`	string	`Operatore`, `Responsabile`, `Amministratore`
`nome`, `cognome`	string	
`attivo`	boolean	un utente non attivo non può entrare

username	password	ruolo
`operatore`	`operatore123`	Operatore
`responsabile`	`responsabile123`	Responsabile
`admin`	`admin123`	Amministratore

2.2 Collection `clients`

Campo	Tipo	Note
`_id`	ObjectId	
`codice_cliente`	string	**codice tessera** (formato `FC-0001`): chiave di ricerca principale usata dalle API
`nome`, `cognome`, `email`, `telefono`	string	l'`email` è la chiave di ricerca alternativa
`saldo_punti`	number	punti attuali
`data_registrazione`	date	
`attivo`	boolean	

Clienti di prova:

_id	Codice	Cliente	Email	Saldo iniziale
`6ae60ea195d55fb2808de666`	`FC-0001`	Luca Ferrari	`luca.ferrari@email.it`	120
`6ae60ea195d55fb2808de667`	`FC-0002`	Giulia Romano	`giulia.romano@email.it`	85
`6ae60ea195d55fb2808de668`	`FC-0003`	Marco Esposito	`marco.esposito@email.it`	850
`6ae60ea195d55fb2808de669`	`FC-0004`	Sara Conti	`sara.conti@email.it`	40
`6ae60ea195d55fb2808de66a`	`FC-0005`	Andrea Marini	`andrea.marini@email.it`	0

Le API 02, 03 e 05 identificano il cliente con `codice_cliente` **oppure** con `email`. Se sono presenti entrambi, vale il codice. Il codice viene ripulito dagli spazi e portato in maiuscolo (`" fc-0001 "`

diventa `FC-0001`).

2.3 Collection `purchases_movements`

Campo	Tipo	Note
`_id`	ObjectId	
`cliente_id`	**string**	l'`_id` del cliente scritto come testo
`tipo`	string	`acquisto`, `riscatto`, `annullamento`
`importo_euro`	number	importo dell'acquisto (0 per i riscatti)
`punti`	number	**con segno**: positivo se aggiunge punti, negativo se li toglie
`data`	date	
`operatore`	string	chi ha eseguito l'operazione
`note`	string	
`movimento_originale_id`	string	solo negli `annullamento`: l'`_id` del movimento annullato
`premio_id`	string	solo nei `riscatto`: l'`_id` del premio

Regole sui segni:

Tipo di movimento	`punti`	`importo_euro`
`acquisto`	+ importo (1 € = 1 punto)	+ importo
`riscatto`	− costo del premio	0
`annullamento` di un acquisto	− punti dell'acquisto	− importo dell'acquisto
`annullamento` di un riscatto	+ punti del riscatto	0

Un movimento è considerato **annullato** quando esiste un movimento di tipo `annullamento` con `movimento_originale_id` uguale al suo `_id`. Il movimento originale **non viene modificato né cancellato**: lo storico resta completo.

Movimenti di partenza:

`_id`	Cliente	Tipo	Importo	Punti	Data	Note
`6ae753591cd6a04d5f8de666`	Luca	acquisto	50	50	2026-05-10	Acquisto prodotti alimentari
`6ae753591cd6a04d5f8de667`	Marco	acquisto	100	100	2026-05-12	Acquisto elettronica
`6ae753591cd6a04d5f8de668`	Sara	acquisto	20	20	2026-05-15	Acquisto abbigliamento
`6ae753591cd6a04d5f8de669`	Luca	annullamento	50	−50	2026-05-11	Storno acquisto errato

Il quarto documento deve contenere anche il campo `"movimento_originale_id": "6ae753591cd6a04d5f8de666"`, così che l'acquisto di Luca del 10 maggio risulti già annullato.

2.4 Collection `rewards_catalog`

Campo	Tipo	Note
`_id`	ObjectId	
`nome`, `descrizione`	string	
`costo_punti`	number	
`disponibile`	boolean	i premi non disponibili non compaiono nel catalogo e non si possono riscattare
`data_inserimento`	date	

_id	Premio	Costo	Disponibile
`6ae6124905bba7c7f88de666`	Buono da 5 €	100	sì
`6ae6124905bba7c7f88de667`	Buono da 20 €	500	sì
`6ae6124905bba7c7f88de668`	Tazza	250	sì
`6ae6124905bba7c7f88de669`	Zaino	800	sì
`6ae6124905bba7c7f88de66a`	Cappellino	90	**no**

2.5 Riportare i dati allo stato iniziale

Dopo i test che modificano i dati si può ripristinare lo stato di partenza dalla console di Compass (pannello `>_MONGOSH`). I comandi cancellano i movimenti creati durante i test e rimettono i saldi:

```
use fidelity_db
db.purchases_movements.deleteMany({ _id: { $nin: [
  ObjectId("6ae753591cd6a04d5f8de666"), ObjectId("6ae753591cd6a04d5f8de667"),
  ObjectId("6ae753591cd6a04d5f8de668"), ObjectId("6ae753591cd6a04d5f8de669") ] } })
db.clients.updateOne({ _id: ObjectId("6ae60ea195d55fb2808de666") }, { $set: { saldo_punti: 120 } })
db.clients.updateOne({ _id: ObjectId("6ae60ea195d55fb2808de667") }, { $set: { saldo_punti: 85 } })
db.clients.updateOne({ _id: ObjectId("6ae60ea195d55fb2808de668") }, { $set: { saldo_punti: 850 } })
db.clients.updateOne({ _id: ObjectId("6ae60ea195d55fb2808de669") }, { $set: { saldo_punti: 40 } })
db.clients.updateOne({ _id: ObjectId("6ae60ea195d55fb2808de66a") }, { $set: { saldo_punti: 0 } })
```

In alternativa, ogni operazione di test si può annullare con l'API 04 (vedi il percorso integrato nel capitolo 10).

---

3. Concetti ricorrenti nei workflow

Webhook con risposta differita. Ogni workflow inizia con un nodo *Webhook* in modalità "Using 'Respond to Webhook' node": il chiamante resta in attesa finché un nodo *Respond to Webhook* non restituisce la risposta. Ogni ramo del workflow termina quindi con un nodo di risposta, anche quelli di errore.

Il pattern "valida, poi decidi". Nelle API 03–06 il primo nodo di codice (`Code - Valida Input`) non risponde da solo: calcola `ok` (vero/falso), `status` (400 o 403), `errore` e `messaggio`. Un nodo IF (`IF - Richiesta Valida?`) manda la richiesta valida avanti e quella non valida a un unico nodo di risposta d'errore, che usa `status` come codice HTTP. Così tutti i controlli sui dati stanno in un solo posto.

Ordine dei controlli. In tutte le API i controlli vanno dal meno al più costoso: prima il formato dei dati, poi il ruolo, poi le letture sul database, infine le scritture. **Il database viene modificato solo dopo che ogni controllo è stato superato.**

Confronto sull'`_id`. L'`_id` dei documenti è un `ObjectId`, ma nelle richieste arriva come testo. Le ricerche per `_id` usano perciò una query `$expr` che converte l'`_id` in stringa (`$toString`) prima del confronto. Per lo stesso motivo `cliente_id` e `movimento_originale_id` sono salvati come stringhe.

`Always Output Data` sui nodi MongoDB di ricerca. Quando una ricerca non trova nulla, un nodo normale non produce alcun risultato e il workflow si ferma senza rispondere. Con questa opzione il nodo emette comunque un elemento vuoto: i controlli successivi (`_id` non vuoto) e i nodi di codice (`filter(m => m._id)`) lo riconoscono come "nessun risultato".

Più risultati → un solo elemento. n8n esegue ogni nodo **una volta per elemento in ingresso**. Una ricerca che restituisce 5 movimenti produrrebbe 5 risposte; per questo i nodi `Code` (`Formatta Risposta`, `Filtra e Formatta`, `Formatta Catalogo`, `Raccogli Clienti`) raccolgono tutti i risultati in un solo elemento prima del nodo di risposta o del nodo successivo.

Date. Il nodo *Set* produce la data come testo ISO (`2026-11-04T11:38:38.473Z`); il nodo MongoDB di inserimento, con l'opzione *Date Fields* = `data`, la salva come vera data di MongoDB, uniforme ai dati di partenza.

Aggiornare il saldo. Il nodo MongoDB *Update* usa `email` come chiave e aggiorna soltanto il campo `saldo_punti`, che gli arriva calcolato dal nodo precedente.

Riferire nodi precedenti. Le espressioni come `$('Code - Valida Input').first().json` leggono l'output di un nodo specifico, anche se non è quello immediatamente a monte. Servono, per esempio, per riusare i dati del webhook dopo una ricerca.

CORS (Flutter Web). Flutter Web gira nel browser, che blocca le chiamate verso un'altra origine (qui `localhost:5678`) se il server non le autorizza. In ogni nodo *Webhook* è quindi impostata l'opzione *Allowed Origins (CORS)* = `*`. Postman non è un browser e non ne ha bisogno: la mancanza di questa opzione si nota solo dall'app.

Lettura dei nodi IF. In tutti i nodi IF l'uscita **vero** è quella in alto e l'uscita **falso** quella in basso. Nelle tabelle dei nodi sono indicate come *Se vero* e *Se falso*.

---

4. API 01 – Login e verifica permessi

Scopo. Verificare username e password e restituire **ruolo, nome e cognome** dell'utente: il ruolo serve a Flutter per mostrare solo le funzioni permesse, nome e cognome per il saluto nel menu.

Richiesta

```
{ "username": "operatore", "password": "operatore123" }
```

Risposta di successo (200)

```
{ "success": true, "ruolo": "Operatore", "nome": "Mario", "cognome": "Rossi", "messaggio": "Login effettuato" }
```

Risposta con credenziali errate (401)

```
{ "success": false, "messaggio": "credenziali non valide" }
```

Schema del flusso

```
Webhook Login
   |
MongoDB - Cerca Utente
   |
Credenziali Valide?
   ├─ vero ──► Respond to Webhook   (200, ruolo)
   └─ falso ─► Respond to Webhook1  (401)
```

I nodi

N.	Nodo	Tipo	Cosa fa
1	`Webhook Login`	Webhook	Riceve la `POST /login`. Il body contiene `username` e `password`.
2	`MongoDB - Cerca Utente`	MongoDB (Find)	Cerca in `users` un documento con `username` e `password` uguali a quelli ricevuti **e** `attivo: true`. Con *Always Output Data* attivo restituisce un elemento vuoto se non trova nessuno.
3	`Credenziali Valide?`	IF	Controlla che il risultato abbia un `_id` non vuoto: se sì, l'utente esiste, la password è giusta ed è attivo.
4	`Respond to Webhook`	Respond	*Se vero*: risponde 200 con `success: true`, `ruolo`, `nome` e `cognome` letti dal database e il messaggio "Login effettuato". Flutter mostra nome e ruolo nel menu.
5	`Respond to Webhook1`	Respond	*Se falso*: risponde 401 con `success: false` e il messaggio "credenziali non valide".

Note. Il workflow non distingue il motivo del rifiuto (utente inesistente, password errata o utente non attivo): per sicurezza la risposta è sempre la stessa. Non esiste un controllo 400 sui campi mancanti: un body vuoto produce comunque 401.

Test API 01 (`POST /webhook/login`)

#	Body	Atteso
01.1	`{"username":"operatore","password":"operatore123"}`	200, `ruolo: "Operatore"`, `nome: "Mario"`, `cognome: "Rossi"`
01.2	`{"username":"responsabile","password":"responsabile123"}`	200, `ruolo: "Responsabile"`
01.3	`{"username":"admin","password":"admin123"}`	200, `ruolo: "Amministratore"`, `nome: "Giuseppe"`, `cognome: "Verdi"`
01.4	`{"username":"operatore","password":"sbagliata"}`	401, "credenziali non valide"
01.5	`{"username":"nessuno","password":"operatore123"}`	401
01.6	`{"username":"admin","password":""}`	401
01.7	`{}`	401

Per provare il controllo `attivo`, in Compass si imposta `attivo: false` su un utente e si ripete il test 01.1: deve rispondere 401. Poi si rimette `attivo: true`.

---

5. API 02 – Ricerca cliente

Scopo. Cercare un cliente tramite il **codice tessera** (`codice_cliente`) oppure l'**email** e restituirne i dati, il saldo punti e gli **ultimi 5 movimenti**.

Richiesta

```
{ "codice_cliente": "FC-0001" }
```

oppure `{ "email": "luca.ferrari@email.it" }`. Se sono presenti entrambi i campi, vale `codice_cliente`.

Risposta di successo (200) – esempio con i dati di partenza:


```
{
  "success": true,
  "cliente": {
    "id": "6ae60ea195d55fb2808de666",
    "codice_cliente": "FC-0001",
    "nome": "Luca",
    "cognome": "Ferrari",
    "email": "luca.ferrari@email.it",
    "saldo_punti": 120
  },
  "movimenti": [
    {
      "_id": "6ae753591cd6a04d5f8de669",
      "cliente_id": "6ae60ea195d55fb2808de666",
      "tipo": "annullamento",
      "importo_euro": 50,
      "punti": -50,
      "data": "2026-05-11T11:00:00.000Z",
      "operatore": "admin",
      "note": "Storno acquisto errato",
      "movimento_originale_id": "6ae753591cd6a04d5f8de666"
    },
    {
      "_id": "6ae753591cd6a04d5f8de666",
      "cliente_id": "6ae60ea195d55fb2808de666",
      "tipo": "acquisto",
      "importo_euro": 50,
      "punti": 50,
      "data": "2026-05-10T10:30:00.000Z",
      "operatore": "operatore",
      "note": "Acquisto prodotti alimentari"
    }
  ],
  "messaggio": "Cliente trovato"
}
```


Se il cliente non ha movimenti, `movimenti` è un array vuoto (risposta 200).

Schema del flusso

```
Webhook Ricerca Cliente
   |
Cliente Indicato?
   ├─ falso ─► Respond - Errore Input (400)
   └─ vero
        |
   MongoDB - Cerca Cliente
        |
   Cliente Esiste?
        ├─ falso ─► Respond - Cliente Non Trovato (404)
        └─ vero
             |
        MongoDB - Cerca Movimenti
             |
        Code - Formatta Risposta
             |
        Respond - Successo (200)
```

I nodi

N.	Nodo	Tipo	Cosa fa
1	`Webhook Ricerca Cliente`	Webhook	Riceve la `POST /ricerca-cliente` con `codice_cliente` oppure `email`.
2	`IF - Cliente Indicato?`	IF	Controlla che almeno uno tra `codice_cliente` ed `email`, privato degli spazi iniziali e finali, non sia vuoto. *Se vero* prosegue, *se falso* va al nodo 3.
3	`Respond - Errore Input`	Respond	Risponde **400** con `INPUT_NON_VALIDO` e il messaggio "Indicare 'codice_cliente' o 'email'".
4	`MongoDB - Cerca Cliente`	MongoDB (Find)	Cerca in `clients` il cliente: per `codice_cliente` (spazi tolti, portato in maiuscolo) se indicato, altrimenti per `email` (spazi tolti). Al massimo un risultato.
5	`IF - Cliente Esiste?`	IF	Controlla che il risultato abbia un `_id`. *Se vero* prosegue, *se falso* va al nodo 6.
6	`Respond - Cliente Non Trovato`	Respond	Risponde **404** con `CLIENTE_NON_TROVATO`.
7	`MongoDB - Cerca Movimenti`	MongoDB (Find)	Legge da `purchases_movements` tutti i movimenti con `cliente_id` uguale all'`_id` del cliente trovato.
8	`Code - Formatta Risposta`	Code	Prende il cliente dal nodo 4 e tutti i movimenti, li ordina dal più recente, tiene i **primi 5**, converte gli `_id` in testo e costruisce l'oggetto di risposta. Gestisce anche il caso "nessun movimento".
9	`Respond - Successo`	Respond	Restituisce l'oggetto costruito dal nodo 8 (**200**).

Note. Il codice tessera non distingue maiuscole e minuscole (è normalizzato), l'email invece è cercata con confronto esatto: `LUCA.FERRARI@email.it` non trova il cliente (404). Il campo `cliente.id`

serve a Flutter per filtrare lo storico dell'API 04 senza una seconda ricerca.

Test API 02 (`POST /webhook/ricerca-cliente`)

Tutti questi test sono **di sola lettura**.

#	Body	Atteso
02.1	`{"codice_cliente":"FC-0001"}`	200, Luca, saldo 120, 2 movimenti: prima l'annullamento dell'11 maggio, poi l'acquisto del 10 maggio
02.2	`{"codice_cliente":"FC-0002"}`	200, Giulia, saldo 85, `movimenti: []`
02.3	`{"codice_cliente":"FC-0003"}`	200, Marco, saldo 850, 1 movimento (acquisto da 100 €)
02.4	`{"email":"luca.ferrari@email.it"}`	200, stesso risultato del 02.1 (ricerca per email)
02.5	`{"codice_cliente":"fc-0001"}`	200, Luca (il codice è portato in maiuscolo)
02.6	`{"codice_cliente":"  FC-0001  "}`	200, Luca (gli spazi vengono tolti)
02.7	`{"codice_cliente":"FC-0002","email":"luca.ferrari@email.it"}`	200, **Giulia**: il codice ha la precedenza sull'email
02.8	`{"codice_cliente":"  ","email":"luca.ferrari@email.it"}`	200, Luca (codice vuoto: si usa l'email)
02.9	`{"codice_cliente":"FC-9999"}`	404, `CLIENTE_NON_TROVATO`
02.10	`{"email":"nessuno@email.it"}`	404, `CLIENTE_NON_TROVATO`
02.11	`{}`	400, `INPUT_NON_VALIDO`, "Indicare 'codice_cliente' o 'email'"
02.12	`{"codice_cliente":"","email":""}`	400
02.13	`{"email":"LUCA.FERRARI@email.it"}`	404 (l'email distingue maiuscole e minuscole)

Per verificare il limite dei 5 movimenti servono almeno 6 movimenti per lo stesso cliente (si ottengono registrando acquisti con l'API 03): la risposta deve contenerne soltanto 5, i più recenti, in ordine decrescente di data.

---

6. API 03 – Registra acquisto

Scopo. Registrare l'acquisto di un cliente (identificato da `codice_cliente` o `email`) applicando la regola **1 € = 1 punto**: aggiornare il saldo e salvare il movimento nello storico.

Richiesta

```
{ "codice_cliente": "FC-0001", "importo_euro": 75, "operatore": "operatore" }
```

Al posto di `codice_cliente` si può usare `email` (se ci sono entrambi vale il codice). `operatore` è facoltativo (valore predefinito: `operatore`).

Risposta di successo (200)


```
{
  "success": true,
  "cliente": {
    "id": "6ae60ea195d55fb2808de666",
    "codice_cliente": "FC-0001",
    "nome": "Luca",
    "cognome": "Ferrari",
    "email": "luca.ferrari@email.it",
    "saldo_punti": 195
  },
  "messaggio": "Acquisto registrato con successo",
  "movimento_id": "ID_DEL_NUOVO_MOVIMENTO",
  "punti_aggiunti": 75,
  "nuovo_saldo": 195
}
```

Schema del flusso

```
Webhook Registra Acquisto
   |
Code - Valida Input
   |
Richiesta Valida?
   ├─ falso ─► Respond - Errore Input (400)
   └─ vero
        |
   MongoDB - Cerca Cliente
        |
   Cliente Esiste?
        ├─ falso ─► Respond - Cliente Non Trovato (404)
        └─ vero
             |
        Set - Calcolo Nuovo Saldo
             |
        MongoDB - Aggiorna Cliente      ◄── prima scrittura
             |
        Set - Prepara Movimento
             |
        MongoDB - Inserisci Movimento   ◄── seconda scrittura
             |
        Respond - Successo (200)
```

I nodi

N.	Nodo	Tipo	Cosa fa
1	`Webhook Registra Acquisto`	Webhook	Riceve la `POST /registra-acquisto`.
2	`Code - Valida Input`	Code	Controlla che almeno uno tra `codice_cliente` ed `email` sia presente, e che `importo_euro` sia un **numero maggiore di 0** (accetta anche un numero scritto come testo, come `"25"`). Calcola i punti con `Math.floor(importo)`: i decimali vengono scartati, quindi 19,99 € valgono 19 punti. Imposta l'operatore predefinito. Normalizza il codice (spazi tolti, maiuscolo) e costruisce il filtro di ricerca (`codice_cliente` se presente, altrimenti `email`). Restituisce `ok`, `messaggio` e i valori normalizzati.
3	`IF - Richiesta Valida?`	IF	Controlla `ok`. *Se vero* prosegue, *se falso* va al nodo 4.
4	`Respond - Errore Input`	Respond	Risponde **400** con `INPUT_NON_VALIDO` e il messaggio dell'errore specifico.
5	`MongoDB - Cerca Cliente`	MongoDB (Find)	Cerca in `clients` il cliente con il filtro costruito dal nodo 2 (codice o email, al massimo uno).

N.	Nodo	Tipo	Cosa fa
6	`IF - Cliente Esiste?`	IF	Controlla che il risultato abbia un `_id`.
7	`Respond - Cliente Non Trovato`	Respond	Risponde **404** con `CLIENTE_NON_TROVATO` e il messaggio "Cliente non trovato".
8	`Set - Calcolo Nuovo Saldo`	Set	Prepara i dati per i passi successivi: `cliente_id`, `codice_cliente`, nome, cognome, email, importo, `punti_da_aggiungere`, `saldo_precedente`, `nuovo_saldo` e `saldo_punti` (saldo attuale + punti), `data_acquisto` (ora attuale), operatore.
9	`MongoDB - Aggiorna Cliente`	MongoDB (Update)	Aggiorna in `clients` il solo campo `saldo_punti`, usando l'`email` come chiave.
10	`Set - Prepara Movimento`	Set	Costruisce il documento del movimento: `cliente_id`, `tipo: "acquisto"`, `importo_euro`, `punti`, `data`, `operatore`, `note: "Acquisto in negozio"`.
11	`MongoDB - Inserisci Movimento`	MongoDB (Insert)	Inserisce il documento in `purchases_movements`. Il campo `data` è convertito in vera data (*Date Fields*).
12	`Respond - Successo`	Respond	Risponde **200** con i dati del cliente, i punti aggiunti, il nuovo saldo e l'`id` del movimento appena creato.

Test API 03 (`POST /webhook/registra-acquisto`)

Errori (non modificano nulla – da eseguire per primi):

#	Body	Atteso
03.E1	`{"codice_cliente":"FC-9999","importo_euro":10}`	404, `CLIENTE_NON_TROVATO`
03.E2	`{}`	400
03.E3	`{"importo_euro":10}`	400, "Indicare 'codice_cliente' o 'email'"
03.E4	`{"codice_cliente":"FC-0001"}`	400, `importo_euro` mancante o non valido
03.E5	`{"codice_cliente":"FC-0001","importo_euro":0}`	400
03.E6	`{"codice_cliente":"FC-0001","importo_euro":-5}`	400
03.E7	`{"codice_cliente":"FC-0001","importo_euro":"abc"}`	400
03.E8	`{"email":"luca.ferrari@email.it","importo_euro":""}`	400
03.E9	`{"email":"nessuno@email.it","importo_euro":10}`	404, `CLIENTE_NON_TROVATO`

Dopo questa serie i saldi e il numero di movimenti devono essere invariati.

Successi (modificano i dati; ognuno restituisce un `movimento_id` da conservare per la pulizia):

#	Body	Atteso
03.S1	`{"codice_cliente":"FC-0001","importo_euro":10,"operatore":"operatore"}`	200, `punti_aggiunti: 10`, `nuovo_saldo: 130`
03.S2	`{"codice_cliente":"FC-0002","importo_euro":20}`	200, `nuovo_saldo: 105`; il movimento ha `operatore: "operatore"` (predefinito). Il saldo di Luca non cambia: prova che il cliente è cercato per codice
03.S3	`{"email":"luca.ferrari@email.it","importo_euro":"25"}`	200, 25 punti, `nuovo_saldo: 155` (ricerca per email)
03.S4	`{"codice_cliente":"fc-0002","importo_euro":19.99}`	200, `punti_aggiunti: 19`, `nuovo_saldo: 124` (codice in minuscolo)
03.S5	`{"email":"giulia.romano@email.it","importo_euro":5,"operatore":"mario"}`	200, movimento con `operatore: "mario"`, `nuovo_saldo: 129`
03.S6	`{"codice_cliente":"  FC-0001  ","importo_euro":5}`	200, `nuovo_saldo: 160`

Controllo in Compass dopo il test 03.S1: nel nuovo documento di `purchases_movements` il campo `data` deve essere una data (non una stringa tra virgolette), `cliente_id` deve essere `6ae60ea195d55fb2808de666`, `tipo` deve essere `acquisto`.

Pulizia. Ogni acquisto di test si annulla con l'API 04 (test 04.C1), usando il suo `movimento_id`. Alla fine Luca torna a 120 e Giulia a 85.

---

7. API 04 – Storico movimenti e annullamento

Scopo. Un solo endpoint, `POST /movimenti`, con due azioni scelte dal campo `azione`:

- **`storico`**: restituisce i movimenti, con filtri opzionali per cliente, periodo e tipo.
- **`annullamento`**: annulla un'operazione, correggendo il saldo del cliente. Solo per l'**Amministratore**.

7.1 Azione `storico`

Richiesta (tutti i filtri sono facoltativi; senza filtri restituisce tutto)

```
{
  "azione": "storico",
  "cliente_id": "6ae60ea195d55fb2808de666",
  "tipo": "acquisto",
  "data_da": "2026-05-01",
  "data_a": "2026-05-31"
}
```

Campo	Significato
`cliente_id`	solo i movimenti di quel cliente (24 caratteri esadecimali)
`tipo`	`acquisto`, `riscatto` o `annullamento`
`data_da`	movimenti dalla data indicata (formato `AAAA-MM-GG`)
`data_a`	movimenti fino alla data indicata, **giorno compreso**

Risposta (200)


```
{
  "success": true,
  "azione": "storico",
  "totale": 1,
  "movimenti": [
    {
      "_id": "6ae753591cd6a04d5f8de666",
      "cliente_id": "6ae60ea195d55fb2808de666",
      "tipo": "acquisto",
      "importo_euro": 50,
      "punti": 50,
      "data": "2026-05-10T10:30:00.000Z",
      "operatore": "operatore",
      "note": "Acquisto prodotti alimentari",
      "annullato": true
    }
  ],
  "messaggio": "Storico movimenti recuperato"
}
```


Ogni movimento ha il campo `annullato` (`true` se esiste un annullamento collegato): in Flutter serve a nascondere il pulsante "Annulla" sulle righe già annullate. Se nessun movimento corrisponde ai filtri la risposta è 200 con `movimenti: []` e il messaggio "Nessun movimento trovato con i filtri indicati".

7.2 Azione `annullamento`

Richiesta


```
{
  "azione": "annullamento",
  "ruolo": "Amministratore",
  "movimento_id": "ID_DEL_MOVIMENTO_DA_ANNULLARE",
  "operatore": "admin"
}
```

Risposta (200)


```
{
  "success": true,
  "azione": "annullamento",
  "movimento_annullato_id": "ID_DEL_MOVIMENTO_DA_ANNULLARE",
  "movimento_annullamento_id": "ID_DEL_NUOVO_ANNULLAMENTO",
  "cliente_id": "6ae60ea195d55fb2808de666",
  "saldo_precedente": 130,
  "punti_stornati": -10,
  "nuovo_saldo_punti": 120,
  "messaggio": "Annullamento effettuato con storno punti"
}
```


`punti_stornati` è **negativo** se si annulla un acquisto (si tolgono punti) e **positivo** se si annulla un riscatto (si restituiscono).

7.3 Schema del flusso


Webhook Movimenti
Code - Valida Input
Richiesta Valida?
falso
Respond - Errore Richiesta (400 oppure 403)
vero
Azione Storico?
vero (STORICO)
Cliente Specificato?
vero
MongoDB - Verifica Cliente
Cliente Esiste?
falso
Respond - Cliente Non Trovato (404)
vero
falso
MongoDB - Cerca Movimenti
Code - Filtra e Formatta
Respond - Successo Storico (200)
falso (ANNULLAMENTO)
MongoDB - Cerca Movimento
Movimento Annullabile?
falso
Respond - Movimento Non Annullabile (404)
vero
MongoDB - Cerca Annullamento Esistente
Gia Annullato?
vero
Respond - Gia Annullato (409)
falso
MongoDB - Cerca Cliente Movimento
Code - Calcolo Nuovo Saldo
MongoDB - Aggiorna Cliente
prima scrittura
Set - Prepara Annullamento
MongoDB - Inserisci Annullamento
seconda scrittura
Respond - Successo Annullamento (200)

7.4 I nodi

Comuni a entrambe le azioni

N.	Nodo	Tipo	Cosa fa
1	`Webhook Movimenti`	Webhook	Riceve la `POST /movimenti`.
2	`Code - Valida Input`	Code	Controlla la richiesta in quest'ordine, fermandosi al primo errore: **(a)** `azione` deve essere `storico` o `annullamento`; **(b)** per lo storico, `cliente_id` (se presente) deve avere 24 caratteri esadecimali e `data_da` / `data_a` (se presenti) devono essere date valide; **(c)** per l'annullamento, `movimento_id` valido, `ruolo` presente, `operatore` presente; **(d)** solo se tutti i campi ci sono, `ruolo` deve essere esattamente `Amministratore`, altrimenti **403**. Prepara anche il filtro per la ricerca (`{ cliente_id }` oppure `{}`).
3	`IF - Richiesta Valida?`	IF	Controlla `ok`. *Se vero* prosegue, *se falso* va al nodo 4.
4	`Respond - Errore Richiesta`	Respond	Risponde con `status` calcolato dal nodo 2: **400** (`INPUT_NON_VALIDO`) oppure **403** (`NON_AUTORIZZATO`, "Solo l'Amministratore può annullare operazioni").
5	`IF - Azione Storico?`	IF	Smista: *se vero* → ramo storico, *se falso* → ramo annullamento (l'unica altra azione possibile, perché le altre sono state respinte al nodo 3).

Ramo storico

N.	Nodo	Tipo	Cosa fa
6	`IF - Cliente Specificato?`	IF	Controlla se è stato indicato un `cliente_id`. *Se vero* va a verificare il cliente, *se falso* salta direttamente alla lettura di tutti i movimenti.
7	`MongoDB - Verifica Cliente`	MongoDB (Find)	Cerca in `clients` il cliente con quell'`_id` (confronto come testo, al massimo uno).
8	`IF - Cliente Esiste?`	IF	Controlla che il risultato abbia un `_id`. *Se vero* prosegue, *se falso* va al nodo 9.
9	`Respond - Cliente Non Trovato`	Respond	Risponde **404** con `CLIENTE_NON_TROVATO`.
10	`MongoDB - Cerca Movimenti`	MongoDB (Find)	Legge da `purchases_movements` i movimenti: solo quelli del cliente se è stato indicato, altrimenti tutti.

N.	Nodo	Tipo	Cosa fa
11	`Code - Filtra e Formatta`	Code	Applica i filtri `tipo`, `data_da` e `data_a` (la data finale include l'intera giornata), calcola per ogni movimento il campo `annullato` (cercando gli annullamenti collegati **su tutti i movimenti letti**, anche se il filtro `tipo` li esclude dal risultato), ordina dal più recente e costruisce la risposta con `totale` e `movimenti`.
12	`Respond - Successo Storico`	Respond	Restituisce la risposta (**200**).

Ramo annullamento

N.	Nodo	Tipo	Cosa fa
13	`MongoDB - Cerca Movimento`	MongoDB (Find)	Cerca in `purchases_movements` il movimento con l'`_id` indicato (confronto come testo).
14	`IF - Movimento Annullabile?`	IF	Due condizioni in AND: il movimento **esiste** e il suo `tipo` **non è** `annullamento`. *Se vero* prosegue, *se falso* va al nodo 15.
15	`Respond - Movimento Non Annullabile`	Respond	Risponde **404** con `MOVIMENTO_NON_ANNULLABILE` ("Il movimento non esiste oppure e gia un annullamento"). Non si può annullare un annullamento.
16	`MongoDB - Cerca Annullamento Esistente`	MongoDB (Find)	Cerca se esiste già un movimento con `movimento_originale_id` uguale all'`_id` del movimento.
17	`IF - Gia Annullato?`	IF	Controlla se la ricerca ha trovato qualcosa. *Se vero* il movimento è già annullato (va al nodo 18); *se falso* si può procedere.
18	`Respond - Gia Annullato`	Respond	Risponde **409** con `MOVIMENTO_GIA_ANNULLATO`. **Impedisce il doppio storno** dei punti.
19	`MongoDB - Cerca Cliente Movimento`	MongoDB (Find)	Legge il cliente a cui appartiene il movimento (dal campo `cliente_id` del movimento).
20	`Code - Calcolo Nuovo Saldo`	Code	Calcola il nuovo saldo: se il movimento è un **acquisto** toglie i suoi punti, altrimenti (riscatto) li **restituisce**. Se il cliente non esiste più, genera un errore (e quindi nessuna scrittura).
21	`MongoDB - Aggiorna Cliente`	MongoDB (Update)	**Prima scrittura**: aggiorna `saldo_punti` del cliente (chiave: `email`).
22	`Set - Prepara Annullamento`	Set	Costruisce il movimento di annullamento: stesso cliente, `tipo: "annullamento"`, importo e punti **di segno opposto**, data attuale, operatore, nota "Annullamento del movimento ..." e `movimento_originale_id`.
23	`MongoDB - Inserisci Annullamento`	MongoDB (Insert)	**Seconda scrittura**: inserisce il documento in `purchases_movements` (con `data` come vera data).
24	`Respond - Successo Annullamento`	Respond	Risponde **200** con gli ID coinvolti, il saldo precedente, i punti stornati e il nuovo saldo.

7.5 Test API 04 (`POST /webhook/movimenti`)

A. Storico (sola lettura) – dati di partenza

#	Body	Atteso
04.A1	`{"azione":"storico"}`	200, `totale: 4`, il più recente per primo
04.A2	`{"azione":"storico","cliente_id":"6ae60ea195d55fb2808de666"}`	200, 2 movimenti di Luca; l'acquisto del 10 maggio ha `annullato: true`
04.A3	`{"azione":"storico","cliente_id":"000000000000000000000000"}`	404, `CLIENTE_NON_TROVATO`
04.A4	`{"azione":"storico","tipo":"annullamento"}`	200, solo il movimento `…de669`
04.A5	`{"azione":"storico","data_da":"2026-05-11"}`	200, 3 movimenti (15, 12 e 11 maggio)
04.A6	`{"azione":"storico","data_a":"2026-05-10"}`	200, solo l'acquisto del 10 maggio (il giorno finale è incluso)
04.A7	`{"azione":"storico","data_da":"2030-01-01"}`	200, `movimenti: []`
04.A8	`{"azione":"storico","cliente_id":"6ae60ea195d55fb2808de666","tipo":"acquisto","data_da":"2026-05-01","data_a":"2026-05-31"}`	200, 1 movimento (acquisto del 10 maggio, `annullato: true`)
04.A9	`{"azione":"storico","cliente_id":"6ae60ea195d55fb2808de667"}`	200, `movimenti: []` (Giulia non ha movimenti)

Richieste non valide (tutte **400**, `INPUT_NON_VALIDO`):

#	Body	Messaggio
04.A10	`{}`	azione mancante o non valida
04.A11	`{"azione":"boh"}`	azione mancante o non valida
04.A12	`{"azione":"storico","cliente_id":"abc"}`	`'cliente_id' non valido`
04.A13	`{"azione":"storico","data_da":"ieri"}`	`'data_da' non valida`
04.A14	`{"azione":"storico","data_a":"domani"}`	`'data_a' non valida`

B. Annullamento – errori (non modificano nulla)

#	Body	Atteso
04.B1	`{"azione":"annullamento","ruolo":"Operatore","movimento_id":"6ae753591cd6a04d5f8de667","operatore":"mario"}`	403, `NON_AUTORIZZATO`
04.B2	come B1 con `"ruolo":"Responsabile"`	403
04.B3	come B1 con `"ruolo":"amministratore"` (minuscolo)	403
04.B4	`{"azione":"annullamento","movimento_id":"6ae753591cd6a04d5f8de667","operatore":"mario"}`	400, "Campo 'ruolo' mancante"
04.B5	`{"azione":"annullamento","ruolo":"Amministratore","movimento_id":"6ae753591cd6a04d5f8de667"}`	400, "Campo 'operatore' mancante"
04.B6	`{"azione":"annullamento","ruolo":"Amministratore","operatore":"mario"}`	400, `movimento_id` mancante o non valido

#	Body	Atteso
04.B7	`{"azione":"annullamento","ruolo":"Amministratore","movimento_id":"123","operatore":"mario"}`	400, `movimento_id` non valido
04.B8	`{"azione":"annullamento","ruolo":"Amministratore","movimento_id":"000000000000000000000000","operatore":"mario"}`	404, `MOVIMENTO_NON_ANNULLABILE`
04.B9	`{"azione":"annullamento","ruolo":"Amministratore","movimento_id":"6ae753591cd6a04d5f8de669","operatore":"mario"}`	404, `MOVIMENTO_NON_ANNULLABILE` (non si annulla un annullamento)
04.B10	`{"azione":"annullamento","ruolo":"Amministratore","movimento_id":"6ae753591cd6a04d5f8de666","operatore":"mario"}`	409, `MOVIMENTO_GIA_ANNULLATO` (l'acquisto di Luca è già annullato nei dati di partenza)

Dopo questa serie i saldi e il numero di movimenti devono essere invariati.

C. Annullamento riuscito (modifica i dati)

Questi test usano i movimenti creati dai test dell'API 03 (`03.S1`, `03.S2`, …): al posto di `ID_MOVIMENTO` si incolla il `movimento_id` ricevuto, **senza lasciare la parola d'esempio**.

#	Body	Atteso
04.C1	`{"azione":"annullamento","ruolo":"Amministratore","movimento_id":"ID_MOVIMENTO_03_S1","operatore":"admin"}`	200, `saldo_precedente: 130`, `punti_stornati: -10`, `nuovo_saldo_punti: 120`. In Compass: Luca a 120 e un nuovo documento `tipo: "annullamento"`, `punti: -10`, `importo_euro: -10`, con `movimento_originale_id`
04.C2	la stessa richiesta del C1	409, `MOVIMENTO_GIA_ANNULLATO`; saldo invariato e **nessun secondo annullamento**
04.C3	`{"azione":"storico","cliente_id":"6ae60ea195d55fb2808de666"}`	il movimento annullato nel C1 ha `annullato: true` e compare la riga di annullamento

I valori attesi del C1 (`saldo_precedente: 130`, `nuovo_saldo_punti: 120`) valgono se, tra gli acquisti dell'API 03, è stato eseguito soltanto il `03.S1`. Se ne sono stati eseguiti altri sullo stesso cliente, `saldo_precedente` e `nuovo_saldo_punti` si spostano della stessa quantità, mentre `punti_stornati` resta uguale ai punti del movimento annullato.

Si ripete il C1 per ogni acquisto di test, fino a riportare Luca a 120 e Giulia a 85.

---

8. API 05 – Catalogo premi e riscatto

Scopo. Un endpoint `POST /premi` riservato all'**Amministratore**, con due azioni:

- `catalogo`: elenca i premi disponibili indicando quali sono riscattabili con il saldo del cliente.
- `riscatto`: riscatta un premio, sottraendo i punti e registrando un movimento nello storico.

8.1 Azione `catalogo`

Richiesta

```
{ "azione": "catalogo", "codice_cliente": "FC-0001", "ruolo": "Amministratore" }
```

Risposta (200)


```
{
  "success": true,
  "azione": "catalogo",
  "cliente": {
    "id": "6ae60ea195d55fb2808de666",
    "codice_cliente": "FC-0001",
    "nome": "Luca",
    "cognome": "Ferrari",
    "email": "luca.ferrari@email.it",
    "saldo_punti": 120
  },
  "totale": 4,
  "premi": [
    { "id": "6ae6124905bba7c7f88de666", "nome": "Buono da 5 €", "descrizione": "…",
"costo_punti": 100, "riscattabile": true, "punti_mancanti": 0 },
    { "id": "6ae6124905bba7c7f88de668", "nome": "Tazza", "descrizione": "…", "costo_punti": 250,
"riscattabile": false, "punti_mancanti": 130 },
    { "id": "6ae6124905bba7c7f88de667", "nome": "Buono da 20 €", "descrizione": "…",
"costo_punti": 500, "riscattabile": false, "punti_mancanti": 380 },
    { "id": "6ae6124905bba7c7f88de669", "nome": "Zaino", "descrizione": "…", "costo_punti": 800,
"riscattabile": false, "punti_mancanti": 680 }
  ],
  "messaggio": "Catalogo premi recuperato"
}
```


I premi sono ordinati per costo crescente; quelli con `disponibile: false` non compaiono.

8.2 Azione `riscatto`

Richiesta

```
{
  "azione": "riscatto",
  "codice_cliente": "FC-0001",
  "ruolo": "Amministratore",
  "premio_id": "6ae6124905bba7c7f88de666",
  "operatore": "admin"
}
```

Al posto di `codice_cliente` si può usare `email` (se ci sono entrambi vale il codice).

Risposta (200)

```
{
  "success": true,
  "azione": "riscatto",
  "movimento_id": "ID_DEL_NUOVO_MOVIMENTO",
  "cliente_id": "6ae60ea195d55fb2808de666",
  "codice_cliente": "FC-0001",
  "premio": { "id": "6ae6124905bba7c7f88de666", "nome": "Buono da 5 €" },
  "saldo_precedente": 120,
  "punti_usati": 100,
  "nuovo_saldo_punti": 20,
  "messaggio": "Premio riscattato con successo"
}
```

8.3 Schema del flusso



8.4 I nodi

Comuni

N.	Nodo	Tipo	Cosa fa
1	`Webhook Premi`	Webhook	Riceve la `POST /premi`.
2	`Code - Valida Input`	Code	Controlla in quest'ordine: **(a)** `azione` è `catalogo` o `riscatto`; **(b)** presente almeno uno tra `codice_cliente` ed `email` (il codice è ripulito e portato in maiuscolo, e determina il filtro di ricerca: codice se presente, altrimenti email); **(c)** `ruolo` presente; **(d)** per il riscatto, `premio_id` valido (24 caratteri esadecimali) e `operatore` presente; **(e)** solo se tutto è presente, `ruolo` deve essere esattamente `Amministratore`, altrimenti **403**.
3	`IF - Richiesta Valida?`	IF	Controlla `ok`.
4	`Respond - Errore Richiesta`	Respond	Risponde **400** (`INPUT_NON_VALIDO`) o **403** (`NON_AUTORIZZATO`, "Solo l'Amministratore può gestire i premi").
5	`MongoDB - Cerca Cliente`	MongoDB (Find)	Cerca il cliente con il filtro del nodo 2 (`codice_cliente` oppure `email`, al massimo uno). Serve a entrambe le azioni.
6	`IF - Cliente Esiste?`	IF	Controlla che il risultato abbia un `_id`.
7	`Respond - Cliente Non Trovato`	Respond	Risponde **404** con `CLIENTE_NON_TROVATO`.
8	`IF - Azione Catalogo ?`	IF	Smista: *se vero* → catalogo, *se falso* → riscatto.

Ramo catalogo

N.	Nodo	Tipo	Cosa fa
9	`MongoDB - Cerca Premi Disponibili`	MongoDB (Find)	Legge da `rewards_catalog` i premi con `disponibile: true`.
10	`Code - Formatta Catalogo`	Code	Per ogni premio calcola `riscattabile` (saldo ≥ costo) e `punti_mancanti`, li ordina per costo e aggiunge i dati del cliente e il totale. Se non ci sono premi risponde con l'elenco vuoto.
11	`Respond - Successo Catalogo`	Respond	Restituisce la risposta (**200**).

Ramo riscatto

N.	Nodo	Tipo	Cosa fa
12	`MongoDB - Cerca Premio`	MongoDB (Find)	Cerca in `rewards_catalog` il premio con l'`_id` indicato (confronto come testo).
13	`IF - Premio Disponibile?`	IF	Due condizioni in AND: il premio **esiste** e ha `disponibile: true`.
14	`Respond - Premio Non Disponibile`	Respond	Risponde **404** con `PREMIO_NON_DISPONIBILE` ("Il premio non esiste oppure non e disponibile").
15	`IF - Saldo Sufficiente?`	IF	Confronta il saldo del cliente con il costo del premio (condizione *saldo* ≥ *costo*; un saldo uguale al costo è sufficiente).
16	`Respond - Saldo Insufficiente`	Respond	Risponde **409** con `SALDO_INSUFFICIENTE` e, nel body, `saldo_punti` e `costo_punti`. Il saldo non viene mai portato sotto zero.
17	`Code - Calcolo Nuovo Saldo`	Code	Calcola `saldo_punti` = saldo precedente − costo. Come ulteriore difesa, genera un errore (e nessuna scrittura) se il costo non è valido o il saldo risultasse insufficiente.
18	`MongoDB - Aggiorna Cliente`	MongoDB (Update)	**Prima scrittura**: aggiorna `saldo_punti` (chiave: `email`).
19	`Set - Prepara Movimento`	Set	Costruisce il movimento: `tipo: "riscatto"`, `punti` **negativi** (costo del premio), `importo_euro: 0`, data attuale, operatore, nota "Riscatto premio: *nome*" e `premio_id`.
20	`MongoDB - Inserisci Movimento`	MongoDB (Insert)	**Seconda scrittura**: inserisce il movimento in `purchases_movements`.
21	`Respond - Successo Riscatto`	Respond	Risponde **200** con l'`id` del movimento, il premio, saldo precedente, punti usati e nuovo saldo.

Un riscatto può essere annullato con l'API 04: i punti vengono restituiti al cliente.

8.5 Test API 05 (`POST /webhook/premi`)

Catalogo (sola lettura)

#	Body	Atteso
05.1	`{"azione":"catalogo","codice_cliente":"FC-0002","ruolo":"Amministratore"}`	200, 4 premi, **nessuno riscattabile** (Giulia ha 85 punti); Buono da 5 € con `punti_mancanti: 15`
05.2	`{"azione":"catalogo","email":"luca.ferrari@email.it","ruolo":"Amministratore"}`	200, solo Buono da 5 € `riscattabile: true` (120 punti); Tazza mancano 130, Buono da 20 € 380, Zaino 680
05.3	`{"azione":"catalogo","codice_cliente":"fc-0003","ruolo":"Amministratore"}`	200, Marco (850 punti): tutti e 4 i premi riscattabili
05.4	`{"azione":"catalogo","codice_cliente":"FC-0005","ruolo":"Amministratore"}`	200, nessun premio riscattabile (saldo 0)
05.5	`{"azione":"catalogo","codice_cliente":"FC-9999","ruolo":"Amministratore"}`	404, `CLIENTE_NON_TROVATO`
05.6	`{"azione":"catalogo","codice_cliente":"FC-0002","ruolo":"Operatore"}`	403, `NON_AUTORIZZATO`

#	Body	Atteso
05.7	`{"azione":"catalogo","codice_cliente":"FC-0002"}`	400, "Campo 'ruolo' mancante"
05.8	`{"azione":"catalogo","ruolo":"Amministratore"}`	400, "Indicare 'codice_cliente' o 'email'"
05.9	`{"azione":"boh","ruolo":"Operatore"}`	400, azione mancante o non valida

Il Cappellino (non disponibile) non deve comparire in nessuna risposta del catalogo.

Riscatto – errori (non modificano nulla)

#	Body	Atteso
05.R1	`{"azione":"riscatto","codice_cliente":"FC-0002","ruolo":"Amministratore","premio_id":"6ae6124905bba7c7f88de666","operatore":"admin"}`	409, `SALDO_INSUFFICIENTE`, `saldo_punti: 85`, `costo_punti: 100` (Buono da 5 €)
05.R2	come R1 con `"premio_id":"6ae6124905bba7c7f88de66a"`	404, `PREMIO_NON_DISPONIBILE` (Cappellino)
05.R3	come R1 con `"premio_id":"000000000000000000000000"`	404, `PREMIO_NON_DISPONIBILE`
05.R4	come R1 con `"ruolo":"Operatore"`	403, `NON_AUTORIZZATO`
05.R5	`{"azione":"riscatto","codice_cliente":"FC-0002","ruolo":"Amministratore","operatore":"admin"}`	400, `premio_id` mancante o non valido
05.R6	`{"azione":"riscatto","codice_cliente":"FC-0002","ruolo":"Amministratore","premio_id":"123","operatore":"admin"}`	400, `premio_id` non valido
05.R7	`{"azione":"riscatto","codice_cliente":"FC-0002","ruolo":"Amministratore","premio_id":"6ae6124905bba7c7f88de666"}`	400, "Campo 'operatore' mancante"
05.R8	`{"azione":"riscatto","codice_cliente":"FC-9999","ruolo":"Amministratore","premio_id":"6ae6124905bba7c7f88de666","operatore":"admin"}`	404, `CLIENTE_NON_TROVATO`
05.R9	`{"azione":"riscatto","ruolo":"Amministratore","premio_id":"6ae6124905bba7c7f88de666","operatore":"admin"}`	400, "Indicare 'codice_cliente' o 'email'"

Dopo questa serie Giulia deve essere ancora a 85 punti.

Riscatto riuscito (modifica i dati)

#	Richiesta	Atteso
05.S1	`{"azione":"riscatto","codice_cliente":"FC-0001","ruolo":"Amministratore","premio_id":"6ae6124905bba7c7f88de666","operatore":"admin"}`	200, `saldo_precedente: 120`, `punti_usati: 100`, `nuovo_saldo_punti: 20`. In Compass: nuovo movimento `tipo: "riscatto"`, `punti: -100`, `importo_euro: 0`, `note: "Riscatto premio: Buono da 5 €"`, con `premio_id`. **Conservare il `movimento_id`.**
05.S2	`{"azione":"riscatto","codice_cliente":"FC-0001","ruolo":"Amministratore","premio_id":"6ae6124905bba7c7f88de668","operatore":"admin"}`	409, `SALDO_INSUFFICIENTE` (ora Luca ha 20 punti, la Tazza ne costa 250)

#	Richiesta	Atteso
05.S3	`{"azione":"storico","cliente_id":"6ae60ea195d55fb2808de666","tipo":"riscatto"}` su `/webhook/movimenti`	200, 1 movimento di tipo riscatto con `annullato: false`
05.S4	`{"azione":"annullamento","ruolo":"Amministratore","movimento_id":"ID_MOVIMENTO_05_S1","operatore":"admin"}` su `/webhook/movimenti`	200, `saldo_precedente: 20`, `punti_stornati: 100`, `nuovo_saldo_punti: 120`
05.S5	la stessa richiesta del S4	409, `MOVIMENTO_GIA_ANNULLATO`
05.S6	la richiesta del S3	il riscatto ha ora `annullato: true`

Le richieste 05.S3, 05.S4 e 05.S6 vanno inviate all'API **04** (`/webhook/movimenti`), non alla 05.

---

9. API 06 – Dashboard

Scopo. Restituire le metriche generali del negozio per la schermata Dashboard, riservata a **Responsabile e Amministratore**.

Richiesta

```
{ "ruolo": "Responsabile" }
```

Risposta (200) – con i dati di partenza:


```
{
  "success": true,
  "totale_clienti": 5,
  "punti_assegnati": 120,
  "punti_utilizzati": 0,
  "valore_acquisti": 120,
  "numero_acquisti": 2,
  "numero_riscatti": 0,
  "clienti_top": [
    { "posizione": 1, "id": "6ae60ea195d55fb2808de668", "nome": "Marco", "cognome": "Esposito",
"email": "marco.esposito@email.it", "saldo_punti": 850 },
    { "posizione": 2, "id": "6ae60ea195d55fb2808de666", "nome": "Luca", "cognome": "Ferrari",
"email": "luca.ferrari@email.it", "saldo_punti": 120 },
    { "posizione": 3, "id": "6ae60ea195d55fb2808de667", "nome": "Giulia", "cognome": "Romano",
"email": "giulia.romano@email.it", "saldo_punti": 85 },
    { "posizione": 4, "id": "6ae60ea195d55fb2808de669", "nome": "Sara", "cognome": "Conti",
"email": "sara.conti@email.it", "saldo_punti": 40 },
    { "posizione": 5, "id": "6ae60ea195d55fb2808de66a", "nome": "Andrea", "cognome": "Marini",
"email": "andrea.marini@email.it", "saldo_punti": 0 }
  ],
  "messaggio": "Dashboard aggiornata"
}
```


Come vengono calcolate le metriche

Campo	Calcolo
`totale_clienti`	numero di documenti in `clients`
`punti_assegnati`	somma dei punti degli **acquisti non annullati**
`punti_utilizzati`	somma dei punti dei **riscatti non annullati** (in valore positivo)
`valore_acquisti`	somma degli importi in euro degli **acquisti non annullati** (arrotondata a 2 decimali)
`numero_acquisti`, `numero_riscatti`	quanti sono i movimenti contati nelle somme precedenti
`clienti_top`	i primi 5 clienti per `saldo_punti`, in ordine decrescente

I movimenti di tipo `annullamento` non vengono mai contati, e **neppure i movimenti che hanno annullato**: la dashboard mostra la situazione reale, non la somma grezza di tutto ciò che è stato registrato. Con i dati di partenza, l'acquisto di Luca da 50 € risulta annullato, quindi restano soltanto i 100 € di Marco e i 20 € di Sara: 120 punti.

Schema del flusso

```
Webhook Dashboard
   |
Code - Valida Input
   |
Richiesta Valida?
   ├─ falso ─► Respond - Errore Richiesta (400 oppure 403)
   └─ vero
        |
   MongoDB - Cerca Clienti
        |
   Code - Raccogli Clienti
        |
   MongoDB - Cerca Movimenti
        |
   Code - Calcola Metriche
        |
   Respond - Successo Dashboard (200)
```

I nodi

N.	Nodo	Tipo	Cosa fa
1	`Webhook Dashboard`	Webhook	Riceve la `POST /dashboard`.
2	`Code - Valida Input`	Code	Controlla che `ruolo` sia presente (altrimenti **400**) e che sia `Responsabile` o `Amministratore` (altrimenti **403**, "Solo Responsabile e Amministratore possono consultare la dashboard").
3	`IF - Richiesta Valida?`	IF	Controlla `ok`.
4	`Respond - Errore Richiesta`	Respond	Risponde **400** o **403** con il `status` calcolato dal nodo 2.
5	`MongoDB - Cerca Clienti`	MongoDB (Find)	Legge **tutti** i clienti di `clients`.
6	`Code - Raccogli Clienti`	Code	Riunisce i clienti in un **solo elemento**. Senza questo passaggio il nodo successivo verrebbe eseguito una volta per cliente e i movimenti risulterebbero duplicati.
7	`MongoDB - Cerca Movimenti`	MongoDB (Find)	Legge **tutti** i movimenti di `purchases_movements`.

N.	Nodo	Tipo	Cosa fa
8	`Code - Calcola Metriche`	Code	Individua i movimenti annullati, esclude annullamenti e movimenti annullati, calcola le somme e i conteggi, ordina i clienti per saldo e tiene i primi 5. Gestisce collection vuote e campi mancanti.
9	`Respond - Successo Dashboard`	Respond	Restituisce le metriche (**200**).

Test API 06 (`POST /webhook/dashboard`)

#	Body	Atteso
06.1	`{"ruolo":"Responsabile"}`	200, `totale_clienti: 5`, `punti_assegnati: 120`, `valore_acquisti: 120`, `numero_acquisti: 2`, `punti_utilizzati: 0`, `numero_riscatti: 0`; top: Marco 850, Luca 120, Giulia 85, Sara 40, Andrea 0
06.2	`{"ruolo":"Amministratore"}`	200, stessi numeri
06.3	`{"ruolo":"Operatore"}`	403, `NON_AUTORIZZATO`
06.4	`{"ruolo":"responsabile"}`	403 (maiuscole e minuscole contano)
06.5	`{}`	400, "Campo 'ruolo' mancante"

I numeri del test 06.1 vanno confrontati con quelli di Compass: il totale dei clienti con i documenti di `clients`, la classifica con i `saldo_punti`.

La dashboard cambia con i dati (test 06.D, modifica i dati; vedi anche il percorso integrato):

#	Operazione	Dashboard attesa
06.D1	API 03: `{"codice_cliente":"FC-0001","importo_euro":30,"operatore":"operatore"}` (conservare il `movimento_id`)	`punti_assegnati: 150`, `valore_acquisti: 150`, `numero_acquisti: 3`; Luca nei top con 150
06.D2	API 05: riscatto del Buono da 5 € per Luca: `{"azione":"riscatto","codice_cliente":"FC-0001","ruolo":"Amministratore","premio_id":"6ae6124905bba7c7f88de666","operatore":"admin"}` (conservare il `movimento_id`)	`punti_utilizzati: 100`, `numero_riscatti: 1`; Luca nei top con 50
06.D3	API 04: annullare il riscatto	`punti_utilizzati: 0`, `numero_riscatti: 0`; Luca a 150
06.D4	API 04: annullare l'acquisto da 30 €	di nuovo i valori del 06.1: 120, 120, 2, 0, 0; Luca a 120

10. Percorso di test integrato

Un unico giro di prova che attraversa tutte le API partendo dai dati iniziali e **li lascia come li ha trovati**. Le richieste che producono un `movimento_id` sono contrassegnate con 📌.

Passo	API (path)	Richiesta	Risultato atteso
1	01 `/login`	`{"username":"admin","password":"admin123"}`	200, `ruolo: "Amministratore"`
2	02 `/ricerca-cliente`	`{"codice_cliente":"FC-0001"}`	saldo 120, 2 movimenti
3	03 `/registra-acquisto` 📌	`{"codice_cliente":"FC-0001","importo_euro":30,"operatore":"operatore"}`	`nuovo_saldo: 150`
4	06 `/dashboard`	`{"ruolo":"Responsabile"}`	150 punti assegnati, 150 €, 3 acquisti
5	05 `/premi`	`{"azione":"catalogo","codice_cliente":"FC-0001","ruolo":"Amministratore"}`	4 premi, solo il Buono da 5 € riscattabile (150 punti)
6	05 `/premi` 📌	`{"azione":"riscatto","codice_cliente":"FC-0001","ruolo":"Amministratore","premio_id":"6ae6124905bba7c7f88de666","operatore":"admin"}`	`nuovo_saldo_punti: 50`
7	06 `/dashboard`	`{"ruolo":"Responsabile"}`	100 punti utilizzati, 1 riscatto
8	04 `/movimenti`	`{"azione":"storico","cliente_id":"6ae60ea195d55fb2808de666"}`	4 movimenti: acquisto e riscatto di oggi (non annullati), più i due di maggio
9	04 `/movimenti`	annullamento del riscatto del passo 6 (`ruolo: "Amministratore"`)	`punti_stornati: 100`, `nuovo_saldo_punti: 150`
10	04 `/movimenti`	annullamento dell'acquisto del passo 3	`punti_stornati: -30`, `nuovo_saldo_punti: 120`
11	04 `/movimenti`	ripetere il passo 10	409, `MOVIMENTO_GIA_ANNULLATO`
12	06 `/dashboard`	`{"ruolo":"Responsabile"}`	di nuovo 120, 120, 2, 0, 0
13	06 `/dashboard`	`{"ruolo":"Operatore"}`	403

Alla fine: Luca a 120, Giulia a 85, e in `purchases_movements` quattro documenti in più (due acquisti/riscatti e i loro due annullamenti), che si possono lasciare come storico dimostrativo o rimuovere con lo script del paragrafo 2.5.

11. Limiti noti e semplificazioni didattiche

Il progetto ammette una sicurezza semplificata, purché sia chiara la distinzione tra autenticazione e autorizzazione. Queste sono le scelte da conoscere (e da saper spiegare):

Tema	Situazione attuale	Come si migliorerebbe
Password	salvate **in chiaro** su MongoDB (ammesso dal testo del progetto)	hash (bcrypt) e confronto lato server
Ruolo	**inviato dal client** nel body: chi chiama le API potrebbe scrivere "Amministratore"	token (per esempio JWT) emesso al login e verificato da ogni API
Login	costruito con interpolazione di testo nella query: caratteri speciali (come le virgolette) in username o password possono rompere la ricerca	costruire la query con `JSON.stringify` come nelle altre API
Due scritture	nelle API 03, 04 e 05 prima si aggiorna il saldo, poi si inserisce il movimento: se la seconda operazione fallisse, il saldo risulterebbe cambiato senza movimento	transazione MongoDB, oppure inserire prima il movimento
Concorrenza	due richieste contemporanee sullo stesso cliente potrebbero leggere lo stesso saldo	aggiornamento atomico con `$inc`
Saldo negativo	annullare un acquisto i cui punti sono già stati spesi può portare il saldo sotto zero	controllo sul saldo prima dell'annullamento
Decimali	l'importo è arrotondato per difetto ai punti interi (19,99 € = 19 punti): il testo del progetto non specifica	regola da concordare
Email e codice	la ricerca per email distingue maiuscole e minuscole; il codice tessera è invece normalizzato in maiuscolo dall'API	salvare e cercare le email in minuscolo
Errori generici	nell'annullamento, se il cliente del movimento non esistesse più, n8n risponderebbe con un errore 500 generico	aggiungere un IF dedicato con risposta 404
Messaggi accorpati	"movimento inesistente" e "annullamento non annullabile" hanno lo stesso errore; così "premio inesistente" e "non disponibile"	separare i controlli
Dashboard	legge tutti i clienti e i movimenti e calcola in un nodo di codice	*Aggregation Pipeline* di MongoDB (`$group`, `$sum`), che il progetto indica come funzionalità avanzata

12. Risoluzione dei problemi

Sintomo	Causa probabile	Rimedio
`400` con "Campo 'movimento_id' mancante o non valido" mentre l'ID sembra scritto	nel body c'è ancora la parola d'esempio (`ID_MOVIMENTO`, `RISCATTO_ID`, ...) invece dell'ID vero	sostituire **tutto il testo tra virgolette** con l'ID copiato dalla risposta precedente
Risposta con messaggio di un altro workflow (per esempio "(catalogo | riscatto)" quando si prova lo storico)	la richiesta è stata inviata al path sbagliato	controllare la barra dell'URL: `movimenti` per l'API 04, `premi` per la 05, ecc.
Postman non riceve risposta o risponde 404 con un messaggio di n8n	il workflow non è pubblicato, oppure si usa `/webhook-test/` senza aver premuto *Execute workflow*	pubblicare il workflow e usare `/webhook/…`
Una modifica al workflow "non ha effetto"	il workflow pubblicato è la versione precedente	ripubblicare dopo ogni modifica
Errore di conflitto tra workflow	due workflow pubblicati con lo stesso path	disattivare o cancellare quello vecchio
Da Postman funziona ma da Flutter Web no (errore di rete / CORS nella console del browser)	manca l'opzione *Allowed Origins (CORS)* nel nodo Webhook	aggiungerla con valore `*` e ripubblicare
Gli acquisti non compaiono nello storico	il nodo di inserimento scrive su un'altra collection (per esempio `purchase_movements`, senza *s*)	controllare il nome nei nodi MongoDB: `purchases_movements`
In Compass non si vedono i documenti appena creati	la lista non è aggiornata	premere *Refresh* e guardare il contatore dei documenti
`data` salvata come testo invece che come data	manca l'opzione *Date Fields = data* nel nodo di inserimento	aggiungerla tra le opzioni del nodo
Il codice di stato non corrisponde (per esempio 200 invece di 404)	il nodo *Respond to Webhook* non ha il *Response Code* impostato	impostarlo nelle opzioni del nodo
Per capire cosa è successo in un workflow	–	scheda **Executions** di n8n: mostra ogni nodo con i dati in ingresso e in uscita

13. Mappa requisiti → API

Requisito del progetto	Dove è soddisfatto
Almeno 3 utenti di prova, uno per ruolo	collection `users` (§2.1), API 01
Login e verifica del ruolo	API 01
Le API verificano i permessi (non solo il frontend)	04 (annullamento), 05 (premi), 06 (dashboard)
Gestire credenziali errate	API 01 (401)
Gestire record inesistenti	404 in 02, 03, 04, 05
Gestire dati mancanti o non validi	400 in 02, 03, 04, 05, 06
Gestire operazioni non autorizzate	403 in 04, 05, 06
Cerca un cliente (per codice tessera o email), mostra saldo e ultimi movimenti	API 02
Il cliente è identificato dal codice tessera o dall'email	API 02, 03, 05 (`codice_cliente` oppure `email`)
Regola 1 € = 1 punto; registrare cliente, data, importo, punti, operatore	API 03
Saldo aggiornato coerentemente	API 03, 04, 05
Storico filtrabile per cliente, periodo e tipologia	API 04, azione `storico`
L'annullamento corregge anche il saldo	API 04, azione `annullamento`
L'amministratore vede i premi compatibili con il saldo	API 05, azione `catalogo`
La richiesta di un premio sottrae i punti e genera un movimento	API 05, azione `riscatto`
Dashboard: clienti, punti assegnati e utilizzati, valore acquisti, clienti con più punti	API 06